

value to process.

Sample Input

630

Sample Output

2

Explanation

Add the holes count for each digit, 6, 3 and 0. Return $1 + 0 + 1 = 2$.

Sample Case 1

Sample Input

1288

Sample Output

4

Explanation

Add the holes count for each digit, 1, 2, 8, 8. Return $0 + 0 + 2 + 2 = 4$.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int num,rem,holes;
5     scanf("%d",&num);
6     while(num>0)
7     {
8         rem=num%10;
9         if(rem==1 || rem==2 || rem==3 || rem==4)
10         {
11             holes++;
12         }
13         else if(rem==8)
14         {
15             holes=holes+2;
16         }
17         else if(rem==6 || rem==9)
18         {
19             holes++;
20         }
21         num=num/10;
22     }
23     printf("%d",holes);
24     return 0;
25 }
```

	Input	Expected	Got	
✓	630	2	2	✓
✓	1288	4	4	✓

Passed all tests! ✓



Constraints

$$1 \leq T \leq 1000$$

$$1 \leq N \leq 10000$$

Sample Input and Output

Input

3

1

6

7

Output

Yes

Yes

No

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int main()
3 {
4     int a,t,i=0,T;
5     scanf("%d",&T);
6     while(i<T)
7     {
8         scanf("%d",&a);
9         t=a/4;
10        if(t%2==0 && a%2==0)
11        {
12            printf("No");
13        }
14        else if(t%2==1 && a%2==1)
15        {
16            printf("No");
17        }
18        else
19        {
20            printf("Yes");
21        }
22        printf("\n");
23        i++;
24    }
25    return 0;
26 }
```

	Input	Expected	Got	
✓	3	Yes	Yes	✓
	1	Yes	Yes	
	6	No	No	
	7			

Passed all tests! ✓

5.00

Flag question

We can rotate digits by 180 degrees to form new digits. When 0, 1, 6, 8, 9 are rotated 180 degrees, they become 0, 1, 9, 8, 6 respectively. When 2, 3, 4, 5 and 7 are rotated 180 degrees, they become invalid. A *confusing number* is a number that when rotated 180 degrees becomes a **different** number with each digit valid.

Example 1:

6 → 9

Input: 6

Output: true

Explanation:

We get 9 after rotating 6, 9 is a valid number and $9 \neq 6$.**Example 2:**

89 → 68

Input: 89

Output: true

Explanation:

We get 68 after rotating 89, 68 is a valid number and $86 \neq 89$.**Example 3:**

11 → 11

Input: 11

Output: false

Explanation:

We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number.

Note:

- $0 \leq N \leq 10^9$
- After the rotation we can ignore leading zeros, for example if after rotation we have 0008 then this number is considered as just 8.

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int main()
3 {
4     int n,x,y=1;
5     scanf("%d",&n);
6     while(n!=0 && y==1)
7     {
8         x=n%10;
9         n=n/10;
10
11         if(x==2 || x==3 || x==4 || x==7)
12         {
13             y++;
14         }
15     }
16     if(y==1)
17     {
18         printf("true");
19     }
20     else
21     {
22         printf("false");
23     }
24     return 0;
25 }

```

	Input	Expected	Got	
✓	6	true	true	✓
✓	89	true	true	✓
✓	25	false	false	✓

Passed all tests! ✓



Question 1

Correct

Marked out of 3.00

Flag question

A set of N numbers (separated by one space) is passed as input to the program. The program must identify the count of numbers where the number is odd number.

Input Format:

The first line will contain the N numbers separated by one space.

Boundary Conditions:

$3 \leq N \leq 50$

The value of the numbers can be from -99999999 to 99999999

Output Format:

The count of numbers where the numbers are odd numbers

Example Input / Output 1:

Input:

5 10 15 20 25 30 35 40 45 50

Output:

5

Explanation:

The numbers meeting the criteria are 5, 15, 25, 35, 45.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int i=0,arr[10],count=0;
5     while(i<10)
6     {
7         scanf("%i",&arr[i]);
8         if(arr[i]%2==1)
9         {
10             count++;
11         }
12         i++;
13     }
14     printf("%d",count);
15     return 0;
16 }
17 }
```

	Input	Expected	Got
✓	5 10 15 20 25 30 35 40 45 50	5	5

Passed all tests! ✓

5

Sample Output 2:

3

Explanation:

For test case 1, N=10.

According to Manish {\$1, \$2, \$3,... \$10} must be distributed.

But as per Manisha only {\$1, \$2, \$3, \$4} coins are enough to purchase any item ranging from \$1 to \$10. Hence minimum is 4. Likewise denominations could also be {\$1, \$2, \$3, \$5}. Hence answer is still 4.

For test case 2, N=5.

According to Manish {\$1, \$2, \$3, \$4, \$5} must be distributed.

But as per Manisha only {\$1, \$2, \$3} coins are enough to purchase any item ranging from \$1 to \$5. Hence minimum is 3. Likewise, denominations could also be {\$1, \$2, \$4}. Hence answer is still 3.

Answer: (penalty regime: 0 %)

```

1 #include<stdio.h>
2 int main()
3 {
4     int num,i;
5     scanf("%d",&num);
6     while(num>0)
7     {
8         num=num/2;
9         i++;
10    }
11    printf("%d",i);
12    return 0;
13
14 }
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	5	3	3	✓
✓	20	5	5	✓
✓	500	9	9	✓
✓	1000	10	10	✓

Passed all tests! ✓

2. $1 + 2 = 3$, observe that this is the max total, and having avoided having exactly $k = 2$ macronutrients.

Sample Input 1

2

1

Sample Output 1

2

Explanation 1

1. Cannot use item 1 because $k = 1$ and $sum \neq k$ has to be avoided at any time.
2. Hence, max total is achieved by $sum = 0 + 2 = 2$.

Sample Case 2

Sample Input For Custom Testing

Sample Input 2

3

3

Sample Output 2

5

Explanation 2

$2 + 3 = 5$, is the best case for maximum nutrients.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     long int n,t,i,nut=0;
5     scanf("%ld %ld",&n,&t);
6     for(i=1;i<=n;i++)
7     {
8         nut=nut+i;
9         if(nut==t)
10        {
11            nut=nut-1;
12        }
13    }
14    printf("%ld",nut%1000000007);
15    return 0;
16 }
17
18
```

	Input	Expected	Got	
✓	2 2	3	3	✓
✓	2 1	2	2	✓
✓	3 3	5	5	✓

Passed all tests! ✓